Project 1: Perception for UDrive

Course: Intro2AV
Format: Group project
Due date: As announced by the instructor

Your role

You are a perception software engineer at **UDrive**, an autonomous-driving startup. Your team has received a recording from a simulated vehicle. The recording contains front-camera images and roof-LiDAR measurements from a vehicle driving through a simulated urban environment. UDrive's planning team needs your software to turn the recorded sensor measurements into useful information about surrounding road users.

For each relevant object, the planning team needs a machine-readable estimate of **what it is** and **where it is relative to the ego vehicle at the measurement time**.

Your task

Develop a ROS 2 perception system that detects **cars and pedestrians** and estimates their positions relative to the ego vehicle. Focus on objects visible in the front camera for which the supplied sensor data support a location estimate. Use both the camera and LiDAR data in your solution.

For this project, a position means an estimate of the object's **center in the horizontal plane**, not the nearest point on its surface. Explain any approximation your method makes when only part of an object is visible. The required outputs are local x and y coordinates in metres.

What you are given

- A short stationary ROS 2 bag for initial testing and a longer driving bag for evaluation.
- Front-camera images, corresponding camera calibration information, roof-LiDAR point clouds, coordinate transforms, and recorded simulation time.
- The ROS 2 output message definitions used for submission.

The camera frame is `front_camera`, and the LiDAR frame is `top_lidar`. Your group should inspect the recording to determine how to use the available data. The bags can be replayed without running the simulator. Use both recordings to test your system.

Data conventions

Camera intrinsics and the declared distortion model are supplied in the recorded `sensor_msgs/CameraInfo` messages. Sensor extrinsics are supplied through `/tf`, including the camera optical-frame rotation. You do not need to perform a new sensor calibration.

Coordinates are in metres. `base_link` and `top_lidar` use x forward, y left, z up. `front_camera` uses optical axes: x right, y down, z forward. The point cloud is already expressed in ROS coordinates; do not apply an additional CARLA-to-ROS axis conversion.

The supplied bags have been tested with **ROS 2 Humble**. Camera images use `bgra8` encoding. The bags include recorded `/clock`; configure time-dependent nodes, including RViz, with `use_sim_time:=true`. Use the original message timestamps when associating sensor data and publishing results.

Required ROS 2 output

Publish results on `/project1/detected_objects` using the supplied `project1_interfaces/msg/DetectedObjectArray` definition. Each message must contain:

Field	Requirement
Message timestamp	The original sensor timestamp for the data used to produce the result.
Message coordinate frame	`base_link`, the ego-vehicle reference frame.
Object class	`car` or `person`. Use `person` for pedestrians.
Confidence	The detector confidence, between 0 and 1.
x	Estimated object-center position in metres, positive forward from the ego reference origin.
y	Estimated object-center position in metres, positive to the ego vehicle's left.

Publish one array for each processed camera/LiDAR pair. Include all valid target detections in that array; publish an empty array when there are none. Do not publish the same object more than once for one sensor timestamp. The `x` and `y` values are signed coordinates, not a single straight-line distance.

The system must process both recordings from beginning to end. Slower-than-real-time playback is acceptable, provided results retain their original sensor timestamps and your README states the playback rate used. Do not evaluate only selected successful frames.

Engineering expectations

Use **C++ or Python with ROS 2**, and calculate your results from the supplied sensor measurements. You are responsible for selecting your methods, inspecting the data, and handling timestamps and coordinate frames. Do not use manually entered object locations, simulator object labels, or answers tied to particular frame numbers.

Handle frames without detections and cases where position cannot be estimated reliably. Do not publish arbitrary coordinates to represent a failed estimate. Existing libraries and pretrained models are allowed; cite their sources and explain your group's implementation and integration work.

What to submit

1. Runnable ROS 2 software

Submit a ROS 2 package or workspace with the source, message definitions, launch files, and configuration needed to run your solution. Include a README with:

- ROS 2 version, dependencies, and instructions for obtaining any required model weights;
- exact build, playback, and launch commands;
- input/output topics, coordinate conventions, and assumptions;
- known limitations and instructions to reproduce your demonstration.

Include a separate result bag for each input sequence, recording your `/project1/detected_objects` output. The instructor must be able to reproduce your run without editing hard-coded paths. Do not resubmit the input bags or build/install directories.

2. One-page technical report

In no more than one page, explain:

- your approach and why you chose it;
- how you checked your detection and location estimates, with at least one numerical result;
- a significant problem your group encountered and how you addressed it;
- an observed failure case or limitation that remains.

Support your claims with evidence. If you estimate reference positions manually, explain your method and its uncertainty.

3. Demonstration video

Submit a short video, approximately **3-5 minutes**, showing your program running on the supplied recording. Show both the visualized results and the ROS 2 output, and explain how a viewer can tell that the reported object locations correspond to the displayed scene. Include representative results and at least one difficult or unsuccessful case. State if playback is slower than real time.

4. Individual contribution summary

Submit a separate table listing each member, their specific contributions, and evidence such as implemented components, tests, experiments, or documentation. This table does not count toward the one-page report limit. Every member should be able to explain their contribution.

Assessment

Detection and position accuracy will be checked against instructor-held reference annotations for selected parts of the supplied recordings. Missed targets, incorrect classes, duplicate detections, false detections, and position errors affect the result. Reference annotations will not be provided as inputs to your solution. The evaluation uses a confidence threshold of 0.25 and matches correctly classified objects within a planar position error of 3 m for cars or 1 m for pedestrians. Each reference object can match at most one prediction at a given timestamp.

Assessment considers detection and position accuracy, correct ROS 2 and coordinate/time handling, reproducibility, validation and failure analysis, communication, and individual contributions. Explain limitations honestly and distinguish demonstrated results from assumptions.